In Class Activity 7: Deep Learning with torch

In Class Activity 7: Deep Learning with torch

This activity uses the **same Earth-observation example and visualizations as the Python version**, but the neural-network workflow is implemented in **R with `torch`**.

Run the environment check first. Then work through the activity from top to bottom.

Learning goals: explain activation functions, compare logistic regression with a neural network, use validation to recognize underfitting and overfitting, and explain epochs, dropout, early stopping, hidden units, and regularization.

torch Environment Check

Before doing any modeling, make sure the R `torch` package and its native libraries are installed correctly.

This R workflow uses `torch` directly.

1. Check your R installation

```
cat("R version:\n")
```

```
## R version:
```

```
print(R.version.string)
```

```
## [1] "R version 4.5.2 (2025-10-31)"
```

```
cat("\nR executable:\n")
```

```
##
## R executable:
```

```
print(R.home())
```

```
## [1] "/Library/Frameworks/R.framework/Resources"
```

```
cat("\nOperating system:\n")
```

```
##
## Operating system:
```

```
print(Sys.info()[c("sysname", "release", "machine")])
```

```
##  sysname  release  machine
## "Darwin" "24.0.0"  "arm64"
```

```
cat("\nWorking directory:\n")
```

```
##
## Working directory:
```

```r
print(getwd())
```

```
## [1] "/Users/abigail/Documents/97_gradSchool/courses/GEOL599_ML/inclass markdowns"
```

2. Install `torch`

Run this once if `torch` is not already installed:

```r
if (!requireNamespace("torch", quietly = TRUE)) {
  install.packages("torch")
}
```

After installing the R package, `torch` may also need to download its native **LibTorch** and **Lantern** components.

If R asks:

```
Do you want to continue? (Yes/no/cancel)
```

enter:

```
Yes
```

and allow both downloads to finish.

If the native-library installation does not start automatically, run this in the same installation session **before loading `torch`**:

```r
options(timeout = 600)

# torch::install_torch()
```

3. Restart RStudio

After `torch`, LibTorch, and Lantern finish installing:

1. In RStudio, click **Session -> Restart R**.
2. Wait for the new R session to start.
3. Return to this R Markdown document.
4. Continue with Section 4.

Do not test `torch` in the same R session in which you installed or updated its native libraries.

4. Load `torch`

In the fresh R session, run:

```r
library(torch)
```

If this completes without an error or RStudio crash, continue.

5. Can R create a tensor?

This is the first functional installation check.

```r
x <- torch_tensor(
  c(1, 2, 3, 4),
  dtype = torch_float32()
)
```

```r
print(x)
```

```
## torch_tensor
##  1
##  2
##  3
##  4
## [ CPUFloatType{4} ]
```

```r
cat("\nSUCCESS: R created a torch tensor.\n")
```

```
##
## SUCCESS: R created a torch tensor.
```

If this prints a tensor, the core `torch` installation is working.

6. Can `torch` perform a calculation?

```r
a <- torch_tensor(
  matrix(
    c(1, 2, 3, 4),
    nrow = 2,
    byrow = TRUE
  ),
  dtype = torch_float32()
)

b <- torch_tensor(
  matrix(
    c(5, 6, 7, 8),
    nrow = 2,
    byrow = TRUE
  ),
  dtype = torch_float32()
)

c_torch <- torch_matmul(a, b)

print(c_torch)
```

```
## torch_tensor
##  19  22
##  43  50
## [ CPUFloatType{2,2} ]
```

```r
cat("\nSUCCESS: torch completed a matrix multiplication.\n")
```

```
##
## SUCCESS: torch completed a matrix multiplication.
```

This calculation runs through R `torch` and LibTorch. It does not call Python.

7. Can `torch` train a tiny neural network?

This is the end-to-end check before the main activity.

```
set.seed(42)
torch_manual_seed(42)

X_check <- matrix(
  rnorm(200),
  ncol = 2
)

y_check <- as.numeric(
  X_check[, 1] + X_check[, 2] > 0
)

check_net <- nn_module(
  "CheckNet",
  initialize = function() {
    self$hidden <- nn_linear(2, 4)
    self$output <- nn_linear(4, 1)
  },
  forward = function(x) {
    x <- self$hidden(x)
    x <- nnf_relu(x)
    self$output(x)
  }
)

check_model <- check_net()

x_check_t <- torch_tensor(
  X_check,
  dtype = torch_float32()
)

y_check_t <- torch_tensor(
  matrix(
    y_check,
    ncol = 1
  ),
  dtype = torch_float32()
)

optimizer <- optim_adam(
  check_model$parameters,
  lr = 0.01
)

loss_fn <- nn_bce_with_logits_loss()

for (epoch in 1:5) {
  check_model$train()

  optimizer$zero_grad()

  logits <- check_model(
```

```
    x_check_t
  )

  loss <- loss_fn(
    logits,
    y_check_t
  )

  loss$backward()

  optimizer$step()
}

cat(
  "\nSUCCESS: torch trained a neural network.\n"
)
```

```
##
## SUCCESS: torch trained a neural network.
```

If this completes, your R `torch` environment is ready for the activity.

Troubleshooting

`there is no package called 'torch'`

Run:

```
install.packages("torch")
```

Then complete any LibTorch/Lantern installation prompts and restart RStudio.

LibTorch or Lantern did not install

Start a fresh R session and, **before calling `library(torch)`**, run:

```
# options(timeout = 600)

# torch::install_torch()
```

Allow the downloads to finish completely.

Then choose **Session -> Restart R** before loading `torch`.

You see `cannot change value of locked binding for '.torch_can_load'`

Do **not** use this diagnostic in this activity:

```
torch::torch_is_installed(recheck = TRUE)
```

That diagnostic can fail while trying to update an internal package value after the namespace has been locked.

Instead, use the direct functional checks in Sections 5-7:

- create a tensor,
- perform a matrix multiplication,
- train a tiny neural network.

Those checks answer the question that matters for this course: **can `torch` actually run?**

`library(torch)` loads, but creating a tensor fails

Restart RStudio completely and try again.

If `torch` was just installed or updated, do not continue using the same R session in which installation occurred.

RStudio crashes when loading `torch`

Restart RStudio and try again in a fresh session.

Do not repeatedly load and unload `torch` in the same R session.

If the problem persists, collect:

```
# sessionInfo()
```

and send the output plus the exact error message to the instructor.

Do I need a GPU?

No. The exercises are designed to run on a CPU.

Deep Learning Activity: Visualizing Deep Learning Concepts

We will use two Landsat spectral bands to distinguish **cotton crop** from **soil with vegetation stubble**. We intentionally use only two central-pixel bands so that the decision surface can be visualized.

The logistic-regression model is the linear baseline. The neural network uses `torch`.

1. Load an Earth-observation dataset

```
library(torch)
library(ggplot2)
```

```
## Use suppressPackageStartupMessages()
## to eliminate package startup
## messages
```

```
RANDOM_STATE <- 7L
set.seed(RANDOM_STATE)
torch_manual_seed(RANDOM_STATE)

url <- "https://raw.githubusercontent.com/dblaskey/ML_Course_Code/main/Data/Landsat_Landcover.csv"

landsat <- read.csv(url, stringsAsFactors = FALSE)

landsat <- landsat[
  landsat$land_cover %in% c(2, 5) &
    !is.na(landsat$pixel_5_band_1) &
    !is.na(landsat$pixel_5_band_2),
  c("land_cover", "pixel_5_band_1", "pixel_5_band_2")
]

landsat$land_cover <- ifelse(
  landsat$land_cover == 5,
  "soil with vegetation stubble",
  "cotton crop"
```

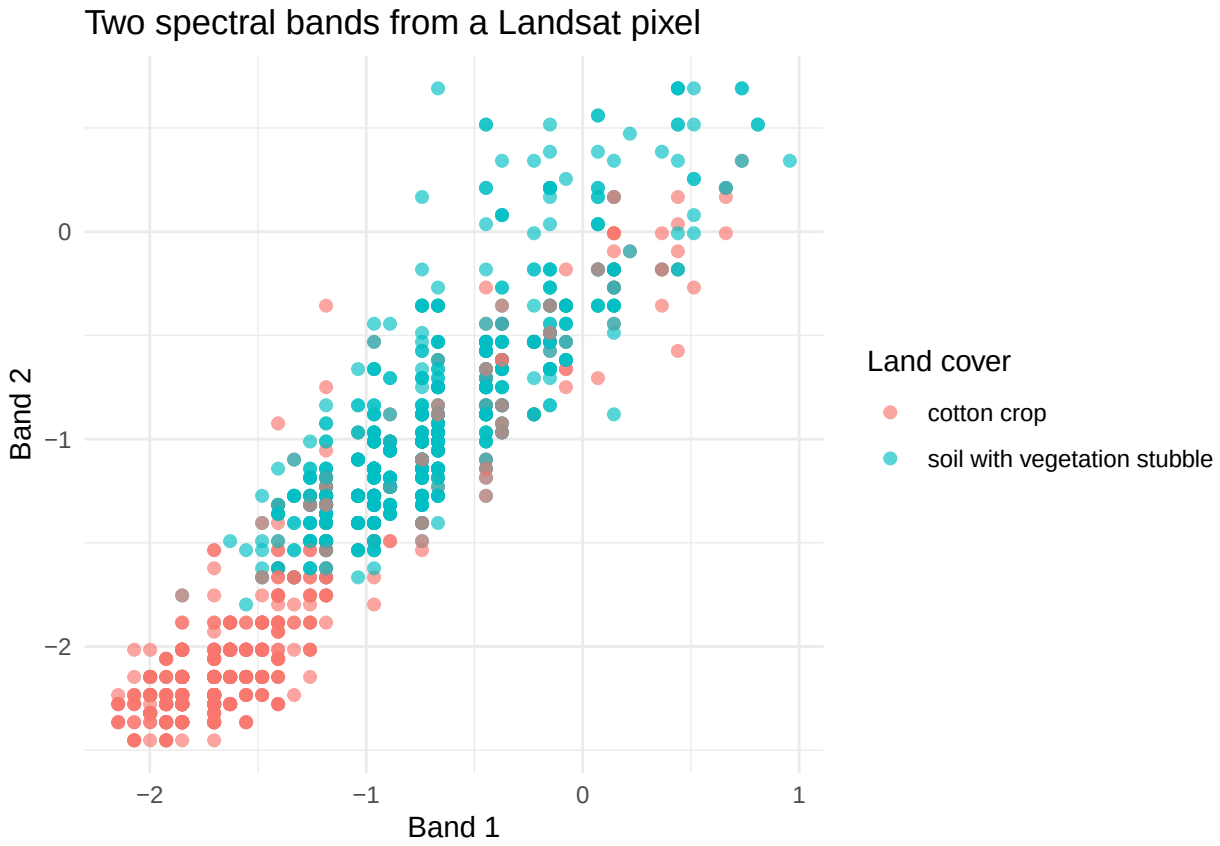
```
)

names(landsat)[names(landsat) == "pixel_5_band_1"] <- "band_1"
names(landsat)[names(landsat) == "pixel_5_band_2"] <- "band_2"

print(table(landsat$land_cover))
```

```
##
##                   cotton crop
##                           703
## soil with vegetation stubble
##                           707
```

```
ggplot(
  landsat,
  aes(x = band_1, y = band_2, color = land_cover)
) +
  geom_point(alpha = 0.65, size = 1.8) +
  labs(
    title = "Two spectral bands from a Landsat pixel",
    x = "Band 1",
    y = "Band 2",
    color = "Land cover"
  ) +
  theme_minimal()
```



2. Split and scale

Neural networks are trained with gradient-based optimization, so predictors should be on comparable scales.

We split the data into training and test sets first, then fit the scaling parameters using **training data only**.

```
# Numeric target: 1 = soil with vegetation stubble, 0 = cotton crop.
y <- ifelse(landsat$land_cover == "soil with vegetation stubble", 1, 0)

X <- as.matrix(landsat[, c("band_1", "band_2")])

# Stratified 75/25 train/test split.
set.seed(RANDOM_STATE)

train_idx_0 <- which(y == 0)
train_idx_1 <- which(y == 1)

n_train_0 <- floor(0.75 * length(train_idx_0))
n_train_1 <- floor(0.75 * length(train_idx_1))

train_idx <- c(
  sample(train_idx_0, n_train_0),
  sample(train_idx_1, n_train_1)
)

test_idx <- setdiff(seq_len(nrow(X)), train_idx)

X_train <- X[train_idx, , drop = FALSE]
X_test <- X[test_idx, , drop = FALSE]
y_train <- y[train_idx]
y_test <- y[test_idx]

# Standardize using training means and standard deviations only.
train_means <- colMeans(X_train)
train_sds <- apply(X_train, 2, sd)

X_train_scaled <- sweep(
  sweep(X_train, 2, train_means, "-"),
  2, train_sds, "/"
)

X_test_scaled <- sweep(
  sweep(X_test, 2, train_means, "-"),
  2, train_sds, "/"
)

cat("Training observations:", nrow(X_train_scaled), "\n")
```

```
## Training observations: 1057
```

```
cat("Test observations:", nrow(X_test_scaled), "\n")
```

```
## Test observations: 353
```

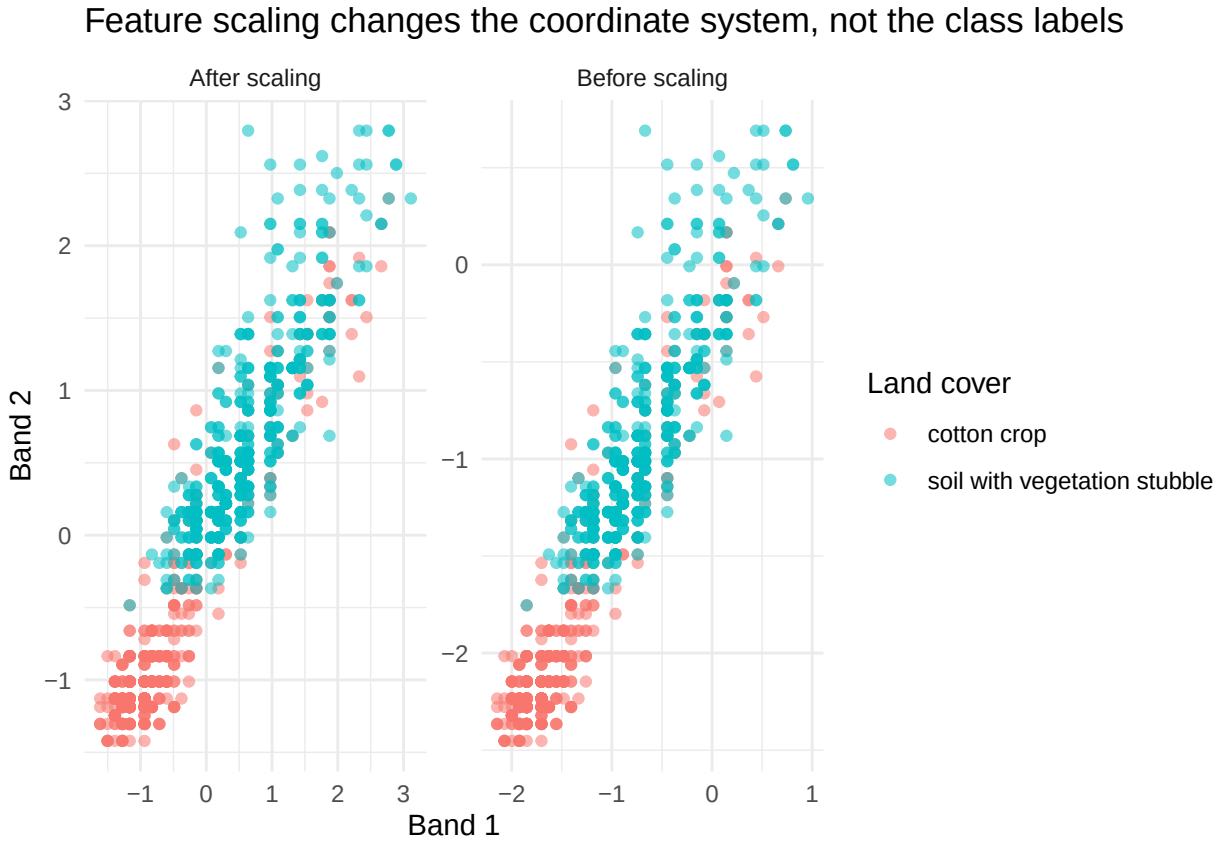
```
raw_train <- data.frame(
  band_1 = X_train[, 1],
  band_2 = X_train[, 2],
```

```
  land_cover = factor(y_train, levels = c(0, 1),
                      labels = c("cotton crop", "soil with vegetation stubble")),
  stage = "Before scaling"
)

scaled_train <- data.frame(
  band_1 = X_train_scaled[, 1],
  band_2 = X_train_scaled[, 2],
  land_cover = factor(y_train, levels = c(0, 1),
                      labels = c("cotton crop", "soil with vegetation stubble")),
  stage = "After scaling"
)

scaling_plot_data <- rbind(raw_train, scaled_train)

ggplot(scaling_plot_data,
       aes(x = band_1, y = band_2, color = land_cover)) +
  geom_point(alpha = 0.55, size = 1.5) +
  facet_wrap(~ stage, scales = "free") +
  labs(
    title = "Feature scaling changes the coordinate system, not the class labels",
    x = "Band 1",
    y = "Band 2",
    color = "Land cover"
  ) +
  theme_minimal()
```



3. Logistic-regression baseline

Logistic regression can learn one linear boundary in the two-feature space. This is our baseline.

We will use the same training/test split as the neural network.

```
logistic_data <- data.frame(
  y = y_train,
  band_1 = X_train_scaled[, 1],
  band_2 = X_train_scaled[, 2]
)

logistic_model <- glm(
  y ~ band_1 + band_2,
  data = logistic_data,
  family = binomial()
)

logistic_probability <- predict(
  logistic_model,
  newdata = data.frame(
    band_1 = X_test_scaled[, 1],
    band_2 = X_test_scaled[, 2]
  ),
  type = "response"
)

logistic_pred <- as.integer(logistic_probability >= 0.5)

accuracy_score <- function(actual, predicted) {
  mean(actual == predicted)
}

binary_log_loss <- function(actual, probability) {
  eps <- 1e-7
  p <- pmin(pmax(probability, eps), 1 - eps)
  -mean(actual * log(p) + (1 - actual) * log(1 - p))
}

cat(
  "Logistic-regression test accuracy:",
  sprintf("%.3f", accuracy_score(y_test, logistic_pred)),
  "\n"
)
```

```
## Logistic-regression test accuracy: 0.892
```

4. Build a small neural network

This model has one hidden layer with eight sigmoid units.

Each hidden unit applies a learned transformation to the input features. The output layer produces a logit; the loss function applies the sigmoid internally for numerical stability.

```
deep_net <- nn_module(
  "DeepNet",
  initialize = function(
```

```
    hidden_units = 8,
    dropout_rate = 0
  ) {
    self$hidden <- nn_linear(2, hidden_units)
    self$dropout <- nn_dropout(p = dropout_rate)
    self$output <- nn_linear(hidden_units, 1)
  },
  forward = function(x) {
    x <- self$hidden(x)
    x <- nnf_sigmoid(x)
    x <- self$dropout(x)
    self$output(x)
  }
)

build_network <- function(
  hidden_units = 8,
  decay = 0.001,
  dropout_rate = 0
) {
  list(
    model = deep_net(
      hidden_units = hidden_units,
      dropout_rate = dropout_rate
    ),
    decay = decay
  )
}

train_torch_model <- function(
  model,
  X_fit,
  y_fit,
  X_validation = NULL,
  y_validation = NULL,
  epochs = 200,
  lr = 0.01,
  decay = 0.001,
  early_stopping = FALSE,
  patience = 10,
  verbose = FALSE
) {
  x_fit_t <- torch_tensor(
    X_fit,
    dtype = torch_float32()
  )

  y_fit_t <- torch_tensor(
    matrix(y_fit, ncol = 1),
    dtype = torch_float32()
  )

  has_validation <- !is.null(X_validation) &&
```

```
  !is.null(y_validation)

if (has_validation) {
  x_validation_t <- torch_tensor(
    X_validation,
    dtype = torch_float32()
  )
  y_validation_t <- torch_tensor(
    matrix(y_validation, ncol = 1),
    dtype = torch_float32()
  )
}

optimizer <- optim_adam(
  model$parameters,
  lr = lr,
  weight_decay = decay
)

loss_fn <- nn_bce_with_logits_loss()

history_rows <- vector("list", epochs)
best_val_loss <- Inf
best_model <- NULL
wait <- 0L
epochs_used <- epochs

for (epoch in seq_len(epochs)) {
  model$train()
  optimizer$zero_grad()

  logits <- model(x_fit_t)
  loss <- loss_fn(logits, y_fit_t)

  loss$backward()
  optimizer$step()

  model$eval()

  with_no_grad({
    train_logits <- model(x_fit_t)
    train_loss <- loss_fn(
      train_logits,
      y_fit_t
    )

    train_probability <- as.numeric(
      torch_sigmoid(
        train_logits
      )$squeeze()$cpu()
    )

    train_accuracy <- mean(
```

```
      y_fit ==
        as.integer(
          train_probability >= 0.5
        )
    )

    if (has_validation) {
      validation_logits <- model(
        x_validation_t
      )

      validation_loss <- loss_fn(
        validation_logits,
        y_validation_t
      )

      validation_probability <- as.numeric(
        torch_sigmoid(
          validation_logits
        )$squeeze()$cpu()
      )

      validation_accuracy <- mean(
        y_validation ==
          as.integer(
            validation_probability >= 0.5
          )
      )

      val_loss_value <- validation_loss$item()
    } else {
      validation_accuracy <- NA_real_
      val_loss_value <- NA_real_
    }
  })

  history_rows[[epoch]] <- data.frame(
    epoch = epoch,
    loss = train_loss$item(),
    accuracy = train_accuracy,
    val_loss = val_loss_value,
    val_accuracy = validation_accuracy
  )

  if (verbose &&
      (epoch == 1 ||
       epoch %% 25 == 0 ||
       epoch == epochs)) {
    cat(
      sprintf(
        "Epoch %d/%d - loss %.4f",
        epoch,
        epochs,
```

```
        train_loss$item()
      ),
      if (has_validation) {
        sprintf(
          " - val_loss %.4f - val_accuracy %.3f",
          val_loss_value,
          validation_accuracy
        )
      } else {
        ""
      },
      "\n"
    )
  }

  if (early_stopping && has_validation) {
    if (val_loss_value < best_val_loss - 1e-3) {
      best_val_loss <- val_loss_value
      best_model <- clone_module(
        model,
        deep = TRUE
      )
      wait <- 0L
    } else {
      wait <- wait + 1L
    }

    if (wait >= patience) {
      epochs_used <- epoch
      break
    }
  }
}

history <- do.call(
  rbind,
  history_rows[
    seq_len(epochs_used)
  ]
)

if (
  early_stopping &&
  !is.null(best_model)
) {
  model <- best_model
}

list(
  model = model,
  history = history,
  epochs_used = epochs_used
)
```

```
}

get_probability <- function(
  model,
  new_data
) {
  x_t <- torch_tensor(
    new_data,
    dtype = torch_float32()
  )

  model$eval()

  with_no_grad({
    as.numeric(
      torch_sigmoid(
        model(x_t)
      )$squeeze()$cpu()
    )
  })
}

set.seed(RANDOM_STATE)
torch_manual_seed(RANDOM_STATE)

nn_parts <- build_network(
  hidden_units = 8,
  decay = 0.001
)

nn_fitted <- train_torch_model(
  nn_parts$model,
  X_train_scaled,
  y_train,
  epochs = 500,
  decay = nn_parts$decay
)

nn_model <- nn_fitted$model

cat("Neural network trained.\n")
```

```
## Neural network trained.
```

Visualize the network architecture

This network has two input features, one hidden layer, and one output probability.

A hidden unit is not a physical thing in the data; it is a learned transformation of the input features.

```
input_nodes <- data.frame(
  layer = "Input",
  node = c("band_1", "band_2"),
  x = 1,
  y = c(2.5, 1.5)
```

```
)

hidden_nodes <- data.frame(
  layer = "Hidden layer",
  node = paste0("h", 1:8),
  x = 2,
  y = seq(4, 0, length.out = 8)
)

output_nodes <- data.frame(
  layer = "Output",
  node = "logit",
  x = 3,
  y = 2
)

network_nodes <- rbind(
  input_nodes,
  hidden_nodes,
  output_nodes
)

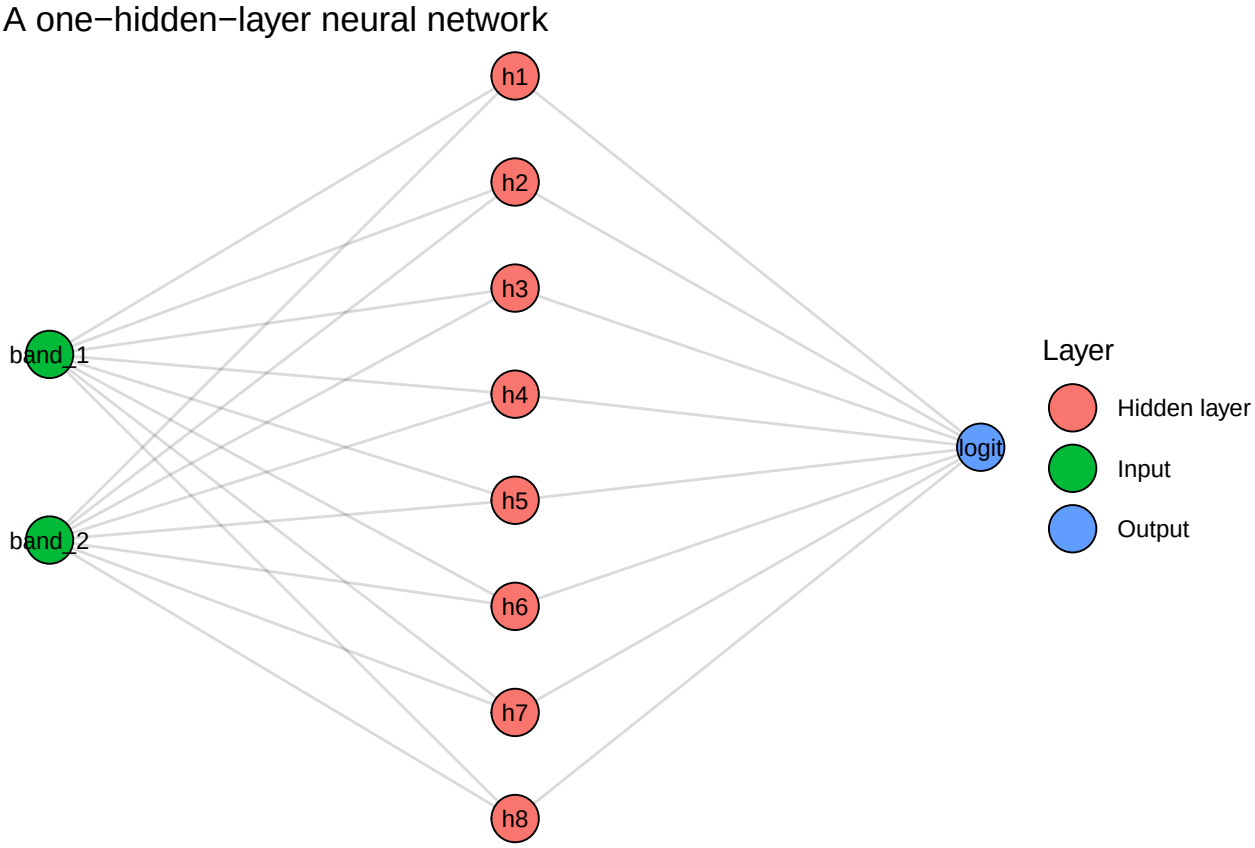
network_edges <- expand.grid(
  input = seq_len(nrow(input_nodes)),
  hidden = seq_len(nrow(hidden_nodes))
)

network_edges$x1 <- input_nodes$x[
  network_edges$input
]
network_edges$y1 <- input_nodes$y[
  network_edges$input
]
network_edges$x2 <- hidden_nodes$x[
  network_edges$hidden
]
network_edges$y2 <- hidden_nodes$y[
  network_edges$hidden
]

network_edges_2 <- data.frame(
  x1 = hidden_nodes$x,
  y1 = hidden_nodes$y,
  x2 = output_nodes$x[1],
  y2 = output_nodes$y[1]
)

ggplot() +
  geom_segment(
    data = network_edges,
    aes(
      x = x1,
      y = y1,
```

```
    xend = x2,
    yend = y2
  ),
  alpha = 0.15
) +
geom_segment(
  data = network_edges_2,
  aes(
    x = x1,
    y = y1,
    xend = x2,
    yend = y2
  ),
  alpha = 0.15
) +
geom_point(
  data = network_nodes,
  aes(
    x = x,
    y = y,
    fill = layer
  ),
  shape = 21,
  color = "black",
  size = 8
) +
geom_text(
  data = network_nodes,
  aes(
    x = x,
    y = y,
    label = node
  ),
  size = 3
) +
scale_x_continuous(
  breaks = c(1, 2, 3),
  labels = c(
    "Input",
    "Hidden",
    "Output"
  )
) +
labs(
  title = "A one-hidden-layer neural network",
  x = NULL,
  y = NULL,
  fill = "Layer"
) +
theme_void() +
theme(
  legend.position = "right"
)
```



Visualize gradient descent and learning rate

Training repeatedly changes weights to reduce loss. This simple example uses one fake parameter and a U-shaped loss curve so we can see the idea.

The visualization illustrates the concept; it is not the actual training path of the neural network above.

```
simulate_descent <- function(learning_rate, start = -3, steps = 12) {
  parameter <- start
  out <- vector("list", steps + 1)

  for (step in 0:steps) {
    loss <- (parameter - 1.5)^2
    out[[step + 1]] <- data.frame(
      step = step,
      parameter = parameter,
      loss = loss,
      learning_rate = paste0("learning rate = ", learning_rate)
    )

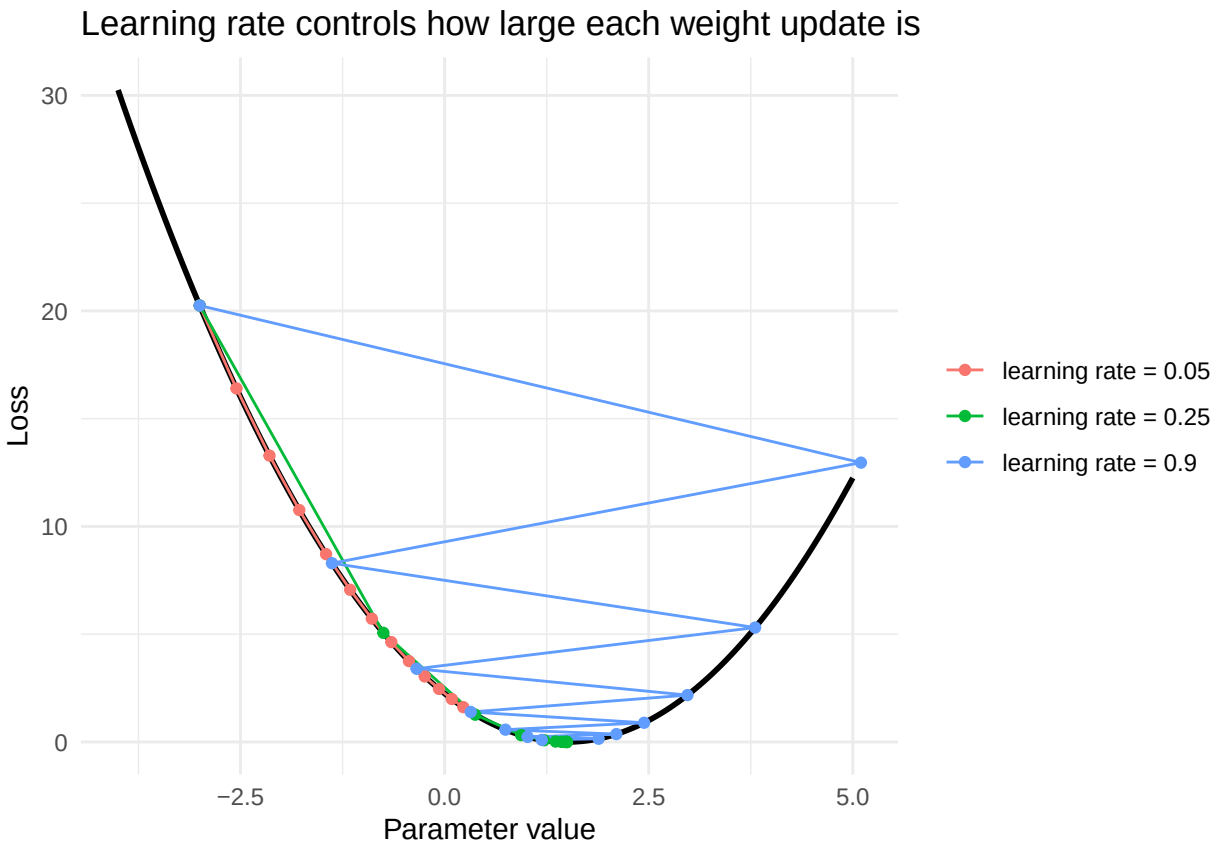
    gradient <- 2 * (parameter - 1.5)
    parameter <- parameter - learning_rate * gradient
  }

  do.call(rbind, out)
}
```

```r
loss_curve <- data.frame(
  parameter = seq(-4, 5, length.out = 500)
)
loss_curve$loss <- (loss_curve$parameter - 1.5)^2

descent_paths <- rbind(
  simulate_descent(0.05),
  simulate_descent(0.25),
  simulate_descent(0.9)
)

ggplot() +
  geom_line(
    data = loss_curve,
    aes(x = parameter, y = loss),
    linewidth = 1
  ) +
  geom_path(
    data = descent_paths,
    aes(
      x = parameter,
      y = loss,
      color = learning_rate,
      group = learning_rate
    )
  ) +
  geom_point(
    data = descent_paths,
    aes(
      x = parameter,
      y = loss,
      color = learning_rate
    )
  ) +
  labs(
    title = "Learning rate controls how large each weight update is",
    x = "Parameter value",
    y = "Loss",
    color = NULL
  ) +
  theme_minimal()
```



5. Use validation to choose model complexity

We compare several hidden-layer sizes on a validation set. The **test set remains untouched**.

Very few units may underfit. More units increase model capacity and can sometimes fit noise.

```
set.seed(RANDOM_STATE)

fit_idx_0 <- sample(
  which(y_train == 0),
  floor(
    0.75 * sum(y_train == 0)
  )
)

fit_idx_1 <- sample(
  which(y_train == 1),
  floor(
    0.75 * sum(y_train == 1)
  )
)

fit_idx <- c(
  fit_idx_0,
  fit_idx_1
)

validation_idx <- setdiff(
```

```
  seq_len(
    nrow(X_train_scaled)
  ),
  fit_idx
)

X_fit <- X_train_scaled[
  fit_idx,
  ,
  drop = FALSE
]

X_validation <- X_train_scaled[
  validation_idx,
  ,
  drop = FALSE
]

y_fit <- y_train[
  fit_idx
]

y_validation <- y_train[
  validation_idx
]

cat(
  "Fit observations:",
  nrow(X_fit),
  "\n"
)
```

```
## Fit observations: 792
```

```
cat(
  "Validation observations:",
  nrow(X_validation),
  "\n"
)
```

```
## Validation observations: 265
```

```
cat(
  "Test observations:",
  nrow(X_test_scaled),
  "\n"
)
```

```
## Test observations: 353
```

```
validation_rows <- list()

for (
  hidden_units in c(
    1, 2, 4, 8, 16
  )
```

```
) {
  set.seed(RANDOM_STATE)
  torch_manual_seed(RANDOM_STATE)

  parts <- build_network(
    hidden_units = hidden_units,
    decay = 0.001
  )

  fitted <- train_torch_model(
    parts$model,
    X_fit,
    y_fit,
    X_validation,
    y_validation,
    epochs = 200,
    decay = parts$decay
  )

  train_probability <- get_probability(
    fitted$model,
    X_fit
  )

  validation_probability <- get_probability(
    fitted$model,
    X_validation
  )

  validation_rows[[
    length(validation_rows) + 1
  ]] <- data.frame(
    hidden_units = hidden_units,
    training_accuracy = accuracy_score(
      y_fit,
      as.integer(
        train_probability >= 0.5
      )
    ),
    validation_accuracy = accuracy_score(
      y_validation,
      as.integer(
        validation_probability >= 0.5
      )
    )
  )
}

validation_results <- do.call(
  rbind,
  validation_rows
)
```

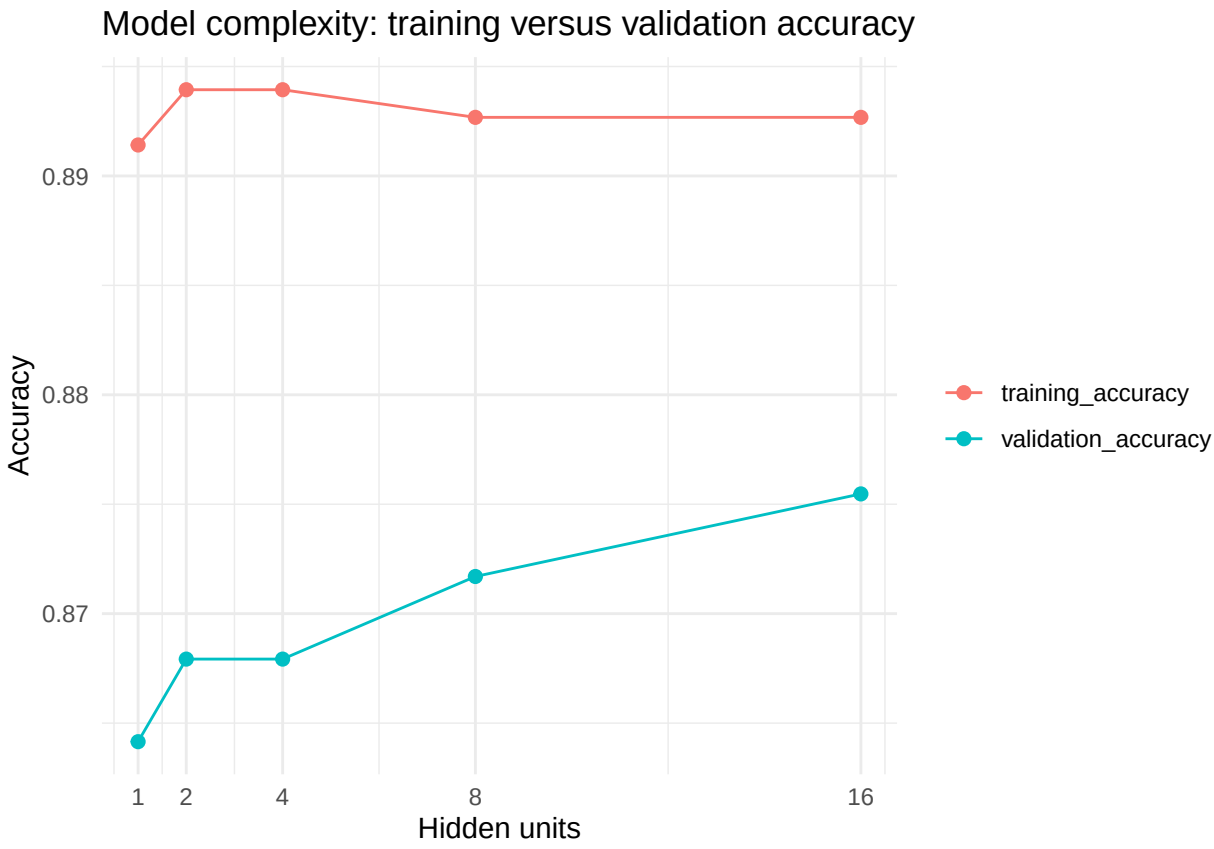
```r
print(validation_results)
```

```
##   hidden_units training_accuracy
## 1            1         0.8914141
## 2            2         0.8939394
## 3            4         0.8939394
## 4            8         0.8926768
## 5           16         0.8926768
##   validation_accuracy
## 1           0.8641509
## 2           0.8679245
## 3           0.8679245
## 4           0.8716981
## 5           0.8754717
```

```r
validation_long <- rbind(
  data.frame(
    hidden_units =
      validation_results$hidden_units,
    data_split =
      "training_accuracy",
    accuracy =
      validation_results$training_accuracy
  ),
  data.frame(
    hidden_units =
      validation_results$hidden_units,
    data_split =
      "validation_accuracy",
    accuracy =
      validation_results$validation_accuracy
  )
)

ggplot(
  validation_long,
  aes(
    x = hidden_units,
    y = accuracy,
    color = data_split
  )
) +
  geom_line() +
  geom_point(size = 2) +
  scale_x_continuous(
    breaks = c(
      1, 2, 4, 8, 16
    )
  ) +
  labs(
    title = "Model complexity: training versus validation accuracy",
    x = "Hidden units",
    y = "Accuracy",
    color = NULL
```

```
  ) +
  theme_minimal()
```



6. What is an epoch?

An **epoch** means the model has made one full pass through the training data.

More epochs can reduce error up to a point, but more training is not automatically better. Eventually, the model can begin fitting patterns that do not generalize.

```
epoch_rows <- list()

for (
  max_epochs in c(
    10, 25, 50, 100, 200, 400
  )
) {
  set.seed(RANDOM_STATE)
  torch_manual_seed(RANDOM_STATE)

  parts <- build_network(
    hidden_units = 8,
    decay = 0.001
  )

  fitted <- train_torch_model(
    parts$model,
    X_fit,
```

```
    y_fit,
    X_validation,
    y_validation,
    epochs = max_epochs,
    decay = parts$decay
  )

  history_df <- fitted$history

  epoch_rows[[
    length(epoch_rows) + 1
  ]] <- data.frame(
    max_epochs = max_epochs,
    training_accuracy =
      tail(
        history_df$accuracy,
        1
      ),
    validation_accuracy =
      tail(
        history_df$val_accuracy,
        1
      ),
    final_training_loss =
      tail(
        history_df$loss,
        1
      ),
    epochs_used =
      fitted$epochs_used
  )
}

epoch_results <- do.call(
  rbind,
  epoch_rows
)

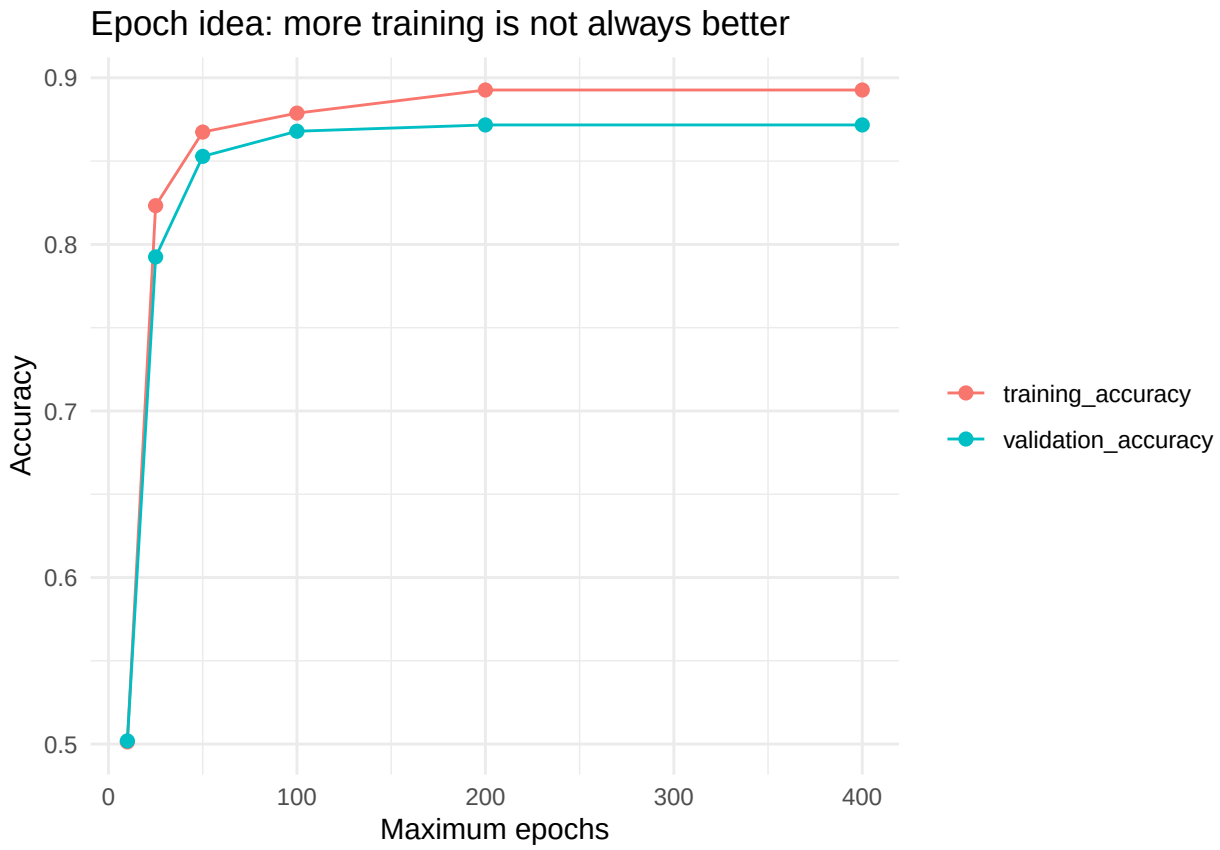
print(epoch_results)
```

```
##   max_epochs training_accuracy
## 1         10         0.5012626
## 2         25         0.8232323
## 3         50         0.8674242
## 4        100         0.8787879
## 5        200         0.8926768
## 6        400         0.8926768
##   validation_accuracy final_training_loss
## 1           0.5018868           0.7026846
## 2           0.7924528           0.6554579
## 3           0.8528302           0.5563425
## 4           0.8679245           0.3675625
## 5           0.8716981           0.3120811
## 6           0.8716981           0.2961764
```

```
##   epochs_used
## 1          10
## 2          25
## 3          50
## 4         100
## 5         200
## 6         400
```

```
epoch_long <- rbind(
  data.frame(
    max_epochs =
      epoch_results$max_epochs,
    data_split =
      "training_accuracy",
    accuracy =
      epoch_results$training_accuracy
  ),
  data.frame(
    max_epochs =
      epoch_results$max_epochs,
    data_split =
      "validation_accuracy",
    accuracy =
      epoch_results$validation_accuracy
  )
)

ggplot(
  epoch_long,
  aes(
    x = max_epochs,
    y = accuracy,
    color = data_split
  )
) +
  geom_line() +
  geom_point(size = 2) +
  labs(
    title = "Epoch idea: more training is not always better",
    x = "Maximum epochs",
    y = "Accuracy",
    color = NULL
  ) +
  theme_minimal()
```



7. Early stopping

Early stopping monitors validation performance during training and stops when validation performance stops improving.

`train_torch_model()` implements early stopping by monitoring validation loss. We monitor validation loss and restore the weights from the best epoch.

```
set.seed(RANDOM_STATE)
torch_manual_seed(RANDOM_STATE)

early_parts <- build_network(
  hidden_units = 8,
  decay = 0.001
)

early_fitted <- train_torch_model(
  early_parts$model,
  X_fit,
  y_fit,
  X_validation,
  y_validation,
  epochs = 500,
  decay = early_parts$decay,
  early_stopping = TRUE,
  patience = 10
)
```

```r
early_stop_history <- early_fitted$history

best_epoch <- which.min(
  early_stop_history$val_loss
)

cat(
  "Epochs used:",
  early_fitted$epochs_used,
  "\n"
)
```

```
## Epochs used: 312
```

```r
cat(
  "Best validation loss:",
  round(
    early_stop_history$val_loss[
      best_epoch
    ],
    3
  ),
  "\n"
)
```

```
## Best validation loss: 0.359
```

```r
cat(
  "Best validation epoch:",
  best_epoch,
  "\n"
)
```

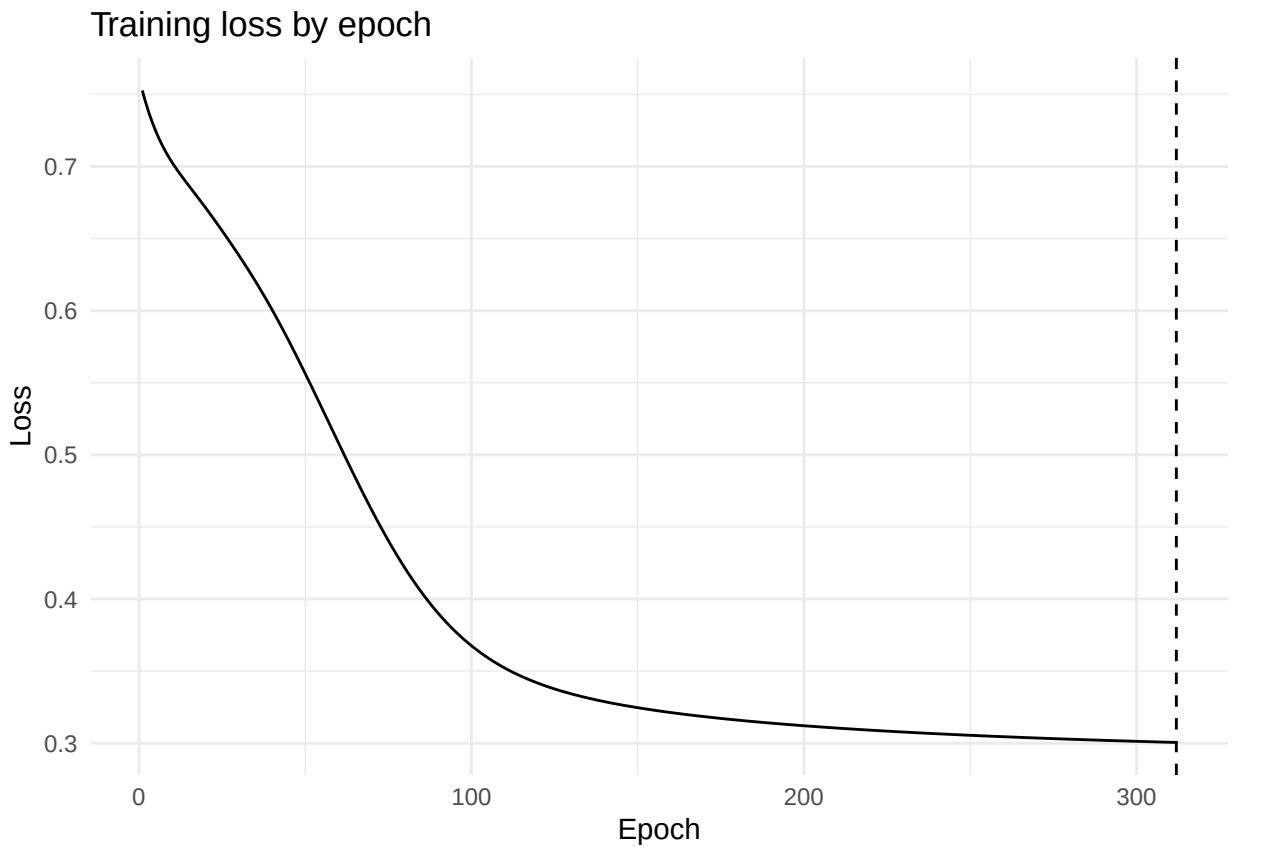
```
## Best validation epoch: 312
```

```r
early_loss_plot <- ggplot(
  early_stop_history,
  aes(
    x = epoch,
    y = loss
  )
) +
  geom_line() +
  geom_vline(
    xintercept = best_epoch,
    linetype = "dashed"
  ) +
  labs(
    title = "Training loss by epoch",
    x = "Epoch",
    y = "Loss"
  ) +
  theme_minimal()

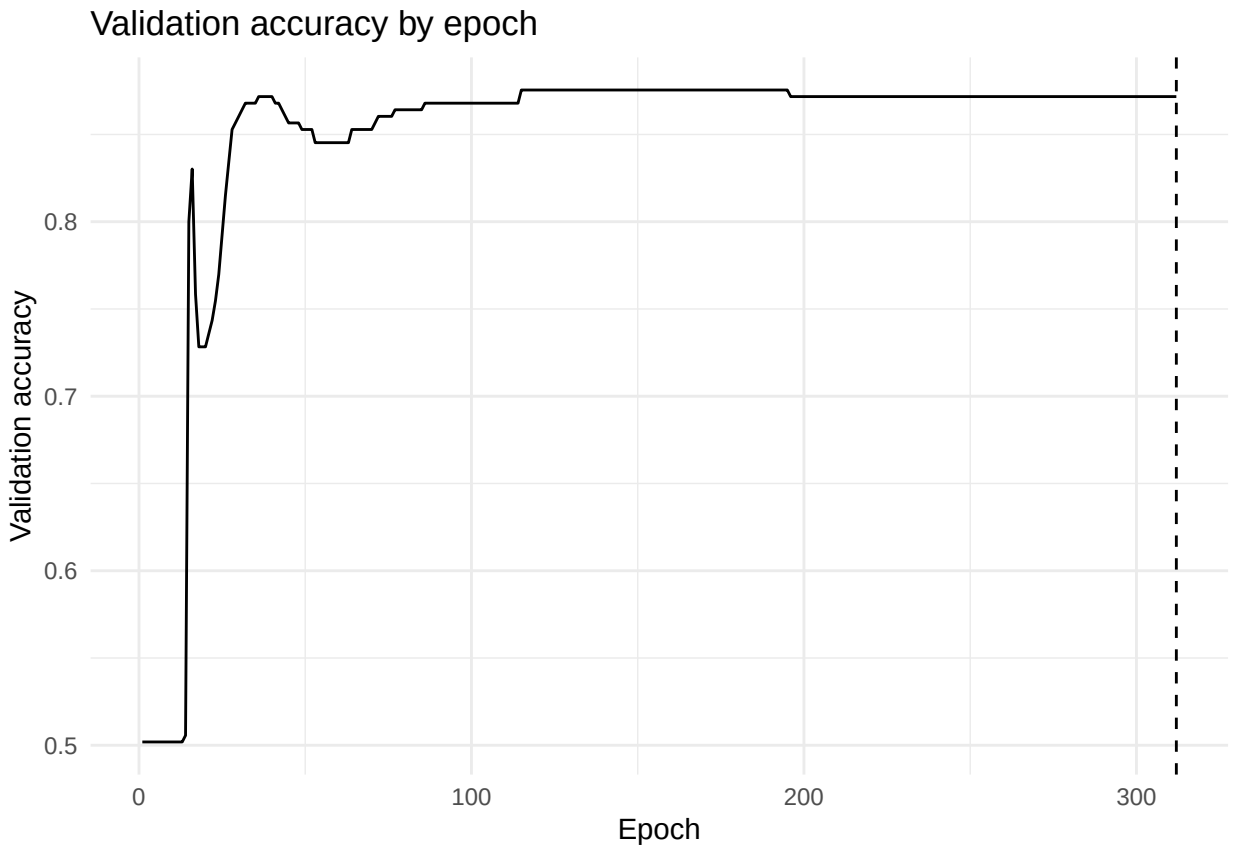
early_accuracy_plot <- ggplot(
  early_stop_history,
```

```
  aes(
    x = epoch,
    y = val_accuracy
  )
) +
  geom_line() +
  geom_vline(
    xintercept = best_epoch,
    linetype = "dashed"
  ) +
  labs(
    title = "Validation accuracy by epoch",
    x = "Epoch",
    y = "Validation accuracy"
  ) +
  theme_minimal()

print(early_loss_plot)
```




```
print(early_accuracy_plot)
```



8. What is dropout?

Dropout is a regularization method. During training, the model randomly turns off some hidden units on each update.

We first visualize the idea, then create a network that includes a dropout layer.

```
set.seed(RANDOM_STATE)

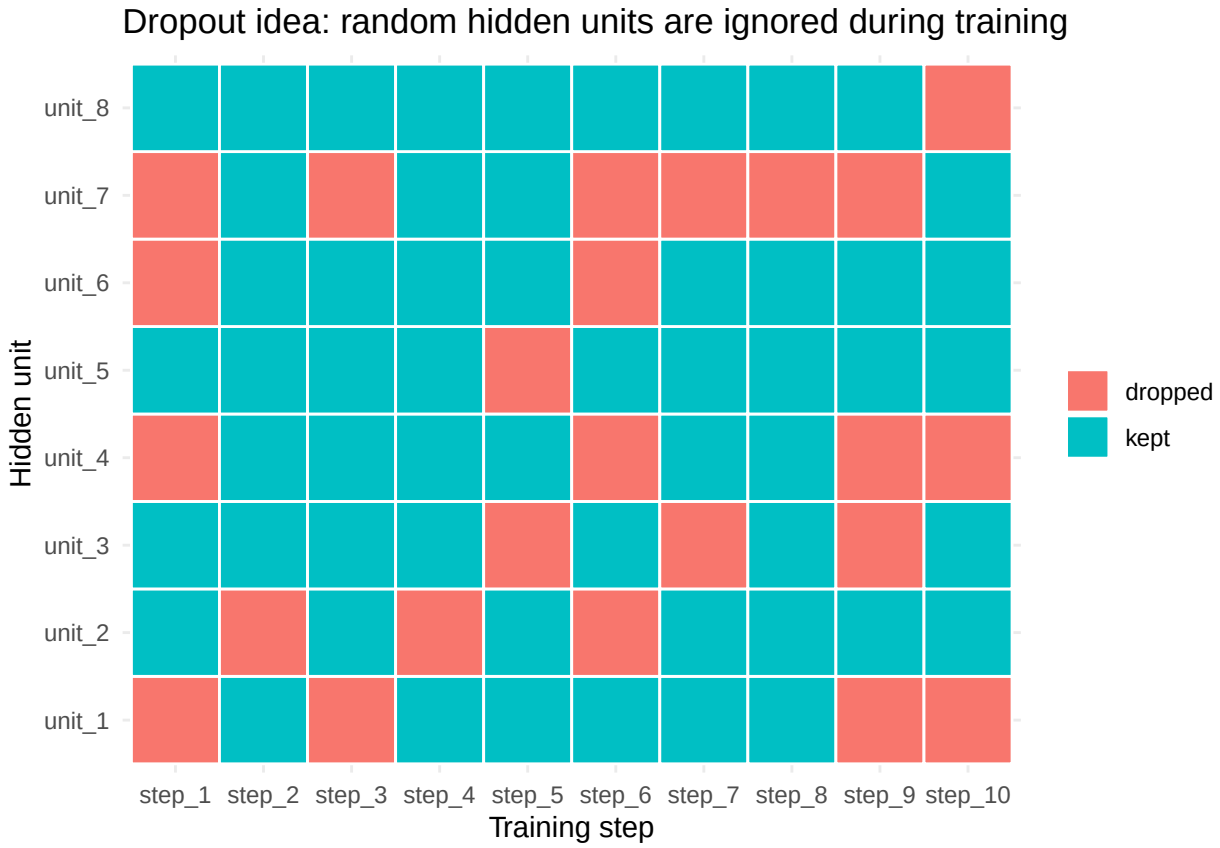
dropout_numeric <- matrix(
  rbinom(
    80,
    size = 1,
    prob = 0.7
  ),
  nrow = 8,
  ncol = 10
)

dropout_demo <- expand.grid(
  unit = paste0(
    "unit_",
    1:8
  ),
  step = paste0(
    "step_",
    1:10
  )
```

```
)

dropout_demo$state <- factor(
  ifelse(
    as.vector(
      t(dropout_numeric)
    ) == 1,
    "kept",
    "dropped"
  ),
  levels = c(
    "dropped",
    "kept"
  )
)

ggplot(
  dropout_demo,
  aes(
    x = step,
    y = unit,
    fill = state
  )
) +
  geom_tile(
    color = "white",
    linewidth = 0.5
  ) +
  labs(
    title = "Dropout idea: random hidden units are ignored during training",
    x = "Training step",
    y = "Hidden unit",
    fill = NULL
  ) +
  theme_minimal()
```




```r
set.seed(RANDOM_STATE)

drop_mask <- runif(
  nrow(network_nodes)
) < 0.7

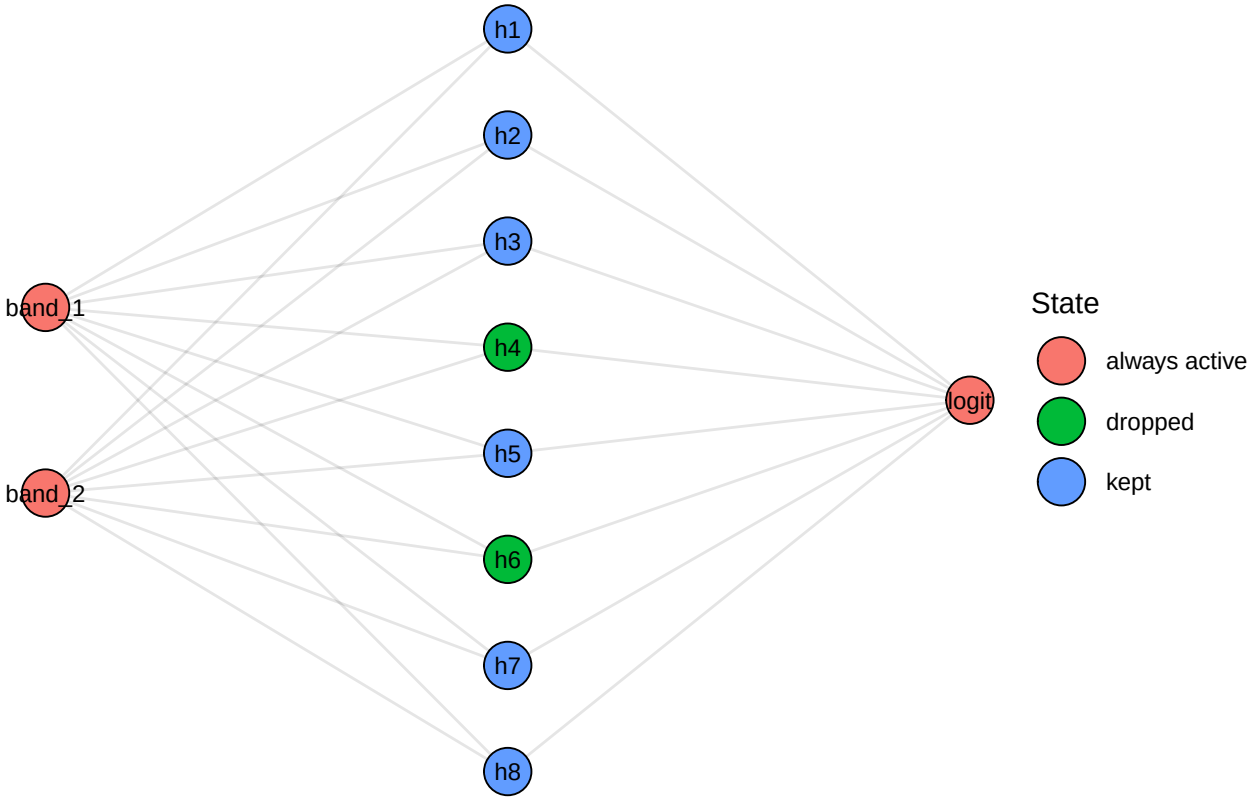
dropout_state <- network_nodes

dropout_state$state <- ifelse(
  dropout_state$layer ==
    "Hidden layer",
  ifelse(
    drop_mask,
    "kept",
    "dropped"
  ),
  "always active"
)

ggplot() +
  geom_segment(
    data = network_edges,
    aes(
      x = x1,
      y = y1,
      xend = x2,
      yend = y2
```

```
  ),
  alpha = 0.10
) +
geom_segment(
  data = network_edges_2,
  aes(
    x = x1,
    y = y1,
    xend = x2,
    yend = y2
  ),
  alpha = 0.10
) +
geom_point(
  data = dropout_state,
  aes(
    x = x,
    y = y,
    fill = state
  ),
  shape = 21,
  color = "black",
  size = 8
) +
geom_text(
  data = dropout_state,
  aes(
    x = x,
    y = y,
    label = node
  ),
  size = 3
) +
scale_x_continuous(
  breaks = c(1, 2, 3),
  labels = c(
    "Input",
    "Hidden",
    "Output"
  )
) +
labs(
  title = "Dropout as a network diagram: some hidden units are ignored",
  x = NULL,
  y = NULL,
  fill = "State"
) +
theme_void() +
theme(
  legend.position = "right"
)
```

Dropout as a network diagram: some hidden units are ignored



9. Evaluate once on the test set

The test set is held back until this point. We use it for final evaluation rather than model selection.

```
nn_probability <- get_probability(
  nn_model,
  X_test_scaled
)

nn_pred <- as.integer(
  nn_probability >= 0.5
)

cat(
  "Neural-network test accuracy:",
  sprintf(
    "%.3f",
    accuracy_score(
      y_test,
      nn_pred
    )
  ),
  "\n"
)
```

```
## Neural-network test accuracy: 0.918
```

```r
cat(
  "Neural-network test log loss:",
  sprintf(
    "%.3f",
    binary_log_loss(
      y_test,
      nn_probability
    )
  ),
  "\n"
)
```

```
## Neural-network test log loss: 0.254
```

```r
cm <- table(
  Observed = factor(
    y_test,
    levels = c(0, 1),
    labels = c(
      "cotton crop",
      "soil with vegetation stubble"
    )
  ),
  Predicted = factor(
    nn_pred,
    levels = c(0, 1),
    labels = c(
      "cotton crop",
      "soil with vegetation stubble"
    )
  )
)

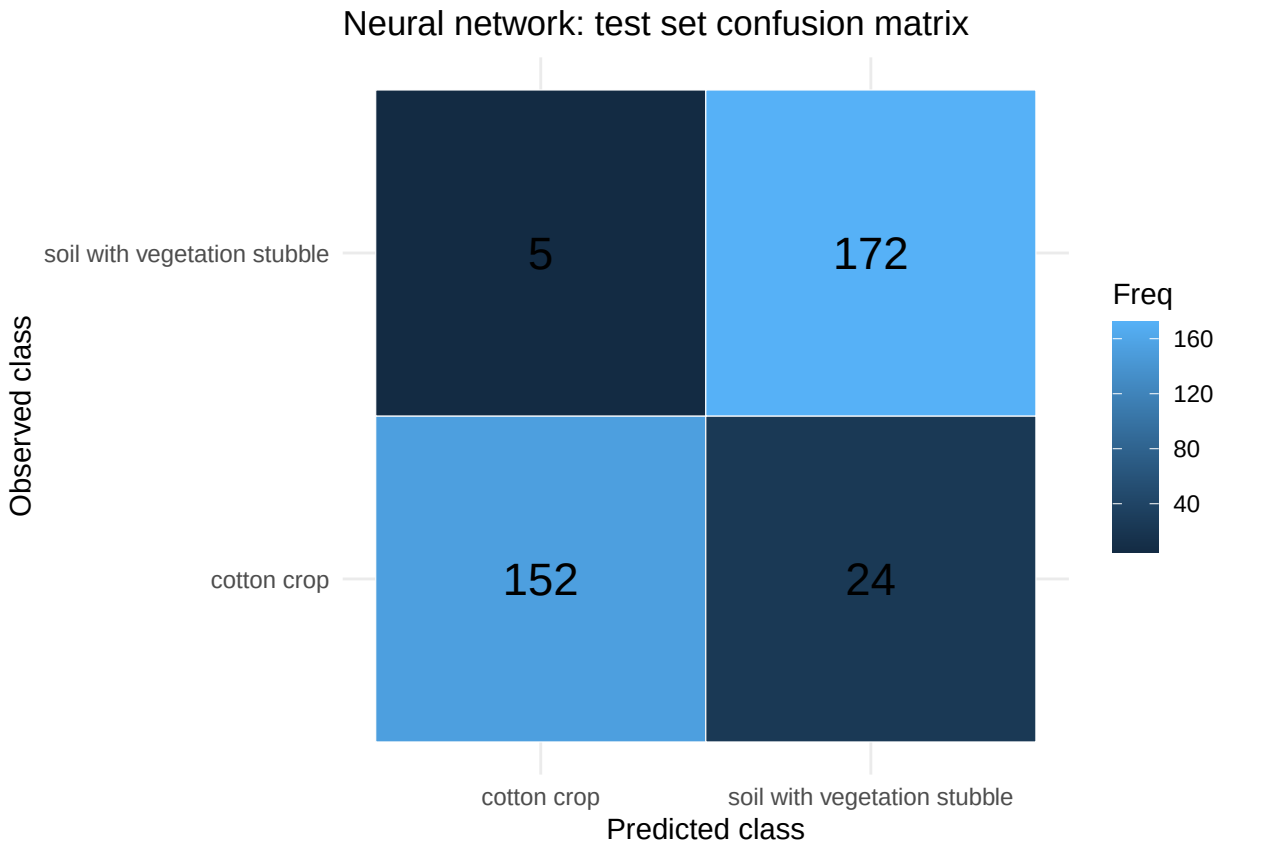
print(cm)
```

```
##                                Predicted
## Observed                        cotton crop
##   cotton crop                           152
##   soil with vegetation stubble            5
##                                Predicted
## Observed                        soil with vegetation stubble
##   cotton crop                                             24
##   soil with vegetation stubble                           172
```

```r
cm_df <- as.data.frame(cm)

ggplot(
  cm_df,
  aes(
    x = Predicted,
    y = Observed,
    fill = Freq
  )
) +
  geom_tile(
```

```
    color = "white"
  ) +
  geom_text(
    aes(label = Freq),
    size = 6
  ) +
  labs(
    title = "Neural network: test set confusion matrix",
    x = "Predicted class",
    y = "Observed class"
  ) +
  theme_minimal()
```



10. Visualize the neural-network decision surface

The background shows predicted probability across a feature grid. The points are the held-out test observations.

The smooth probability surface contains more information than the 0.5 decision boundary alone.

The probability surface is the smooth background. The decision boundary is only the contour where the model crosses 0.5. The probability surface tells us about confidence; the boundary tells us where the predicted class changes.

```
make_prediction_grid <- function(
  n = 250
) {
  xx <- seq(
    min(landsat$band_1),
```

```
    max(landsat$band_1),
    length.out = n
  )

  yy <- seq(
    min(landsat$band_2),
    max(landsat$band_2),
    length.out = n
  )

  grid <- expand.grid(
    band_1 = xx,
    band_2 = yy
  )

  grid_scaled <- sweep(
    sweep(
      as.matrix(grid),
      2,
      train_means,
      "-"
    ),
    2,
    train_sds,
    "/"
  )

  list(
    xx = xx,
    yy = yy,
    grid = grid,
    grid_scaled = grid_scaled
  )
}

prediction_grid <- make_prediction_grid()

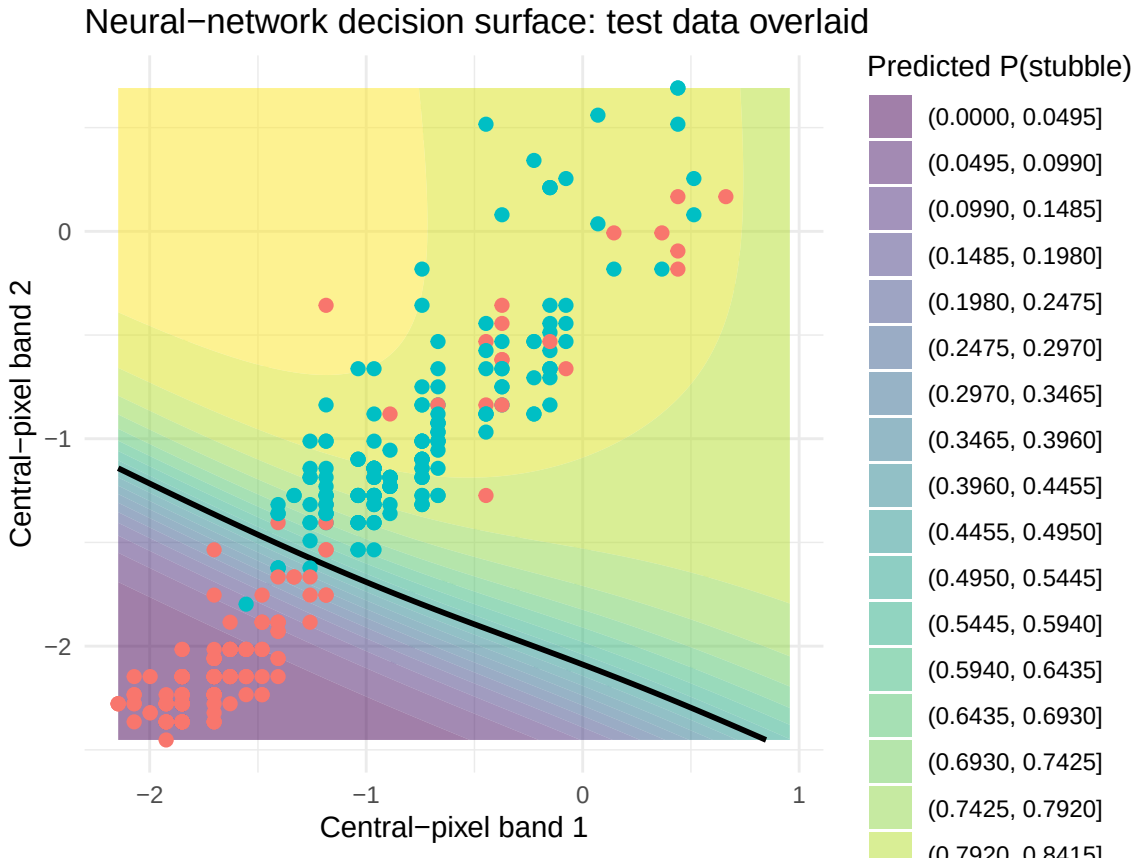
grid_probability <- get_probability(
  nn_model,
  prediction_grid$grid_scaled
)

grid_data <- prediction_grid$grid
grid_data$probability <-
  grid_probability

test_plot_data <- data.frame(
  band_1 = X_test[, 1],
  band_2 = X_test[, 2],
  land_cover = factor(
    y_test,
    levels = c(0, 1),
    labels = c(
```

```
      "cotton crop",
      "soil with vegetation stubble"
    )
  )
)

ggplot(
  grid_data,
  aes(
    x = band_1,
    y = band_2,
    z = probability
  )
) +
  geom_contour_filled(
    bins = 20,
    alpha = 0.5
  ) +
  geom_contour(
    breaks = 0.5,
    color = "black",
    linewidth = 1
  ) +
  geom_point(
    data = test_plot_data,
    aes(
      x = band_1,
      y = band_2,
      color = land_cover
    ),
    inherit.aes = FALSE,
    size = 2
  ) +
  labs(
    title = "Neural-network decision surface: test data overlaid",
    x = "Central-pixel band 1",
    y = "Central-pixel band 2",
    color = "Observed test class",
    fill = "Predicted P(stubble)"
  ) +
  theme_minimal()
```



Compare the linear and nonlinear decision surfaces

```
xx_grid <- prediction_grid$grid

xx_grid$logistic_probability <- predict(
  logistic_model,
  newdata = data.frame(
    band_1 =
      prediction_grid$grid_scaled[, 1],
    band_2 =
      prediction_grid$grid_scaled[, 2]
  ),
  type = "response"
)

xx_grid$neural_probability <- get_probability(
  nn_model,
  prediction_grid$grid_scaled
)

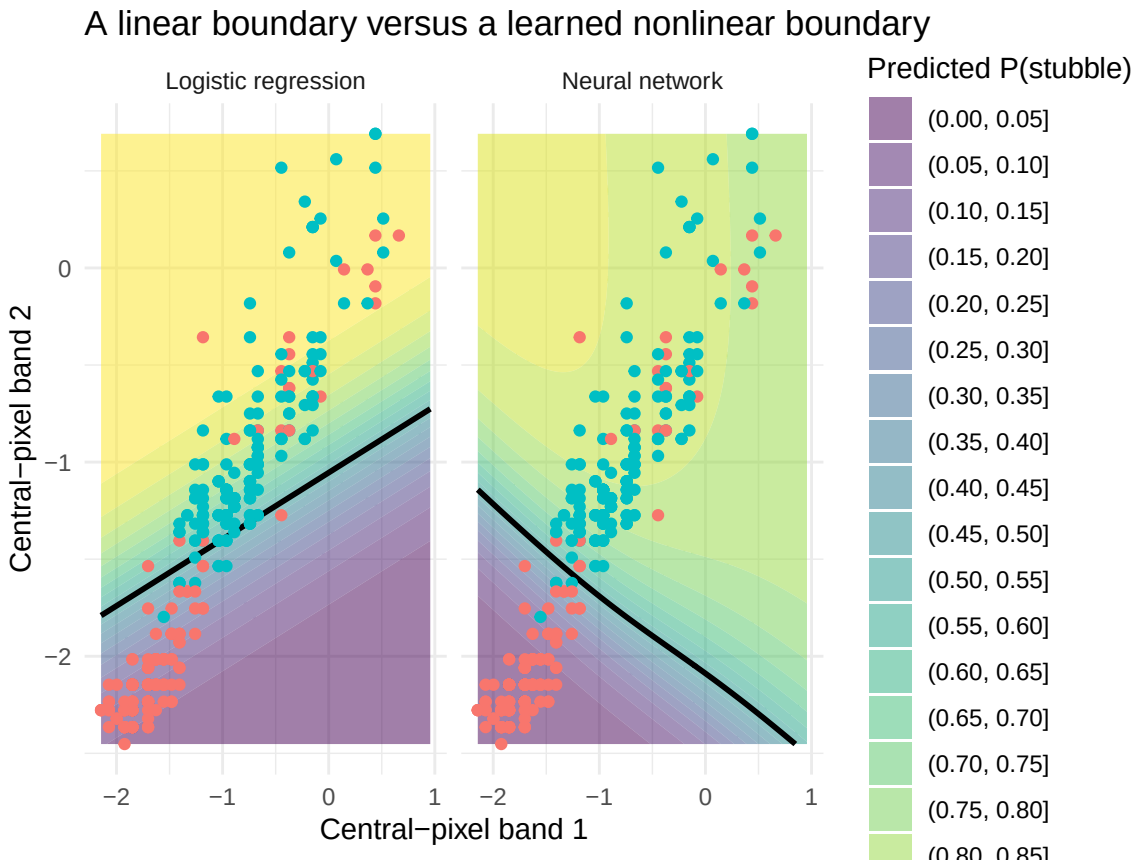
logistic_surface <- data.frame(
  band_1 = xx_grid$band_1,
  band_2 = xx_grid$band_2,
  probability =
    xx_grid$logistic_probability,
  model = "Logistic regression"
```

```
)

neural_surface <- data.frame(
  band_1 = xx_grid$band_1,
  band_2 = xx_grid$band_2,
  probability =
    xx_grid$neural_probability,
  model = "Neural network"
)

comparison_surface <- rbind(
  logistic_surface,
  neural_surface
)

ggplot(
  comparison_surface,
  aes(
    x = band_1,
    y = band_2,
    z = probability
  )
) +
  geom_contour_filled(
    bins = 20,
    alpha = 0.5
  ) +
  geom_contour(
    breaks = 0.5,
    color = "black",
    linewidth = 1
  ) +
  geom_point(
    data = test_plot_data,
    aes(
      x = band_1,
      y = band_2,
      color = land_cover
    ),
    inherit.aes = FALSE,
    size = 1.5
  ) +
  facet_wrap(
    ~ model
  ) +
  labs(
    title = "A linear boundary versus a learned nonlinear boundary",
    x = "Central-pixel band 1",
    y = "Central-pixel band 2",
    fill = "Predicted P(stubble)",
    color = "Observed class"
  ) +
  theme_minimal()
```



Visualize hidden-unit activation maps

Each hidden unit learns its own transformation of the two input features.

Because the hidden layer uses **sigmoid** activation, we can calculate each unit's activation directly from its learned weights and biases.

```
hidden_layer <- nn_model$hidden

hidden_weights <- as_array(
  hidden_layer$weight
)

hidden_bias <- as.numeric(
  as_array(
    hidden_layer$bias
  )
)

sigmoid <- function(z) {
  1 / (1 + exp(-z))
}

hidden_activation_values <- outer(
  prediction_grid$grid_scaled[, 1],
  hidden_weights[, 1],
  "*"
```

```
) +
  outer(
    prediction_grid$grid_scaled[, 2],
    hidden_weights[, 2],
    "*"
  )

hidden_activation_values <- sweep(
  hidden_activation_values,
  2,
  hidden_bias,
  "+"
)

hidden_activation_values <- sigmoid(
  hidden_activation_values
)

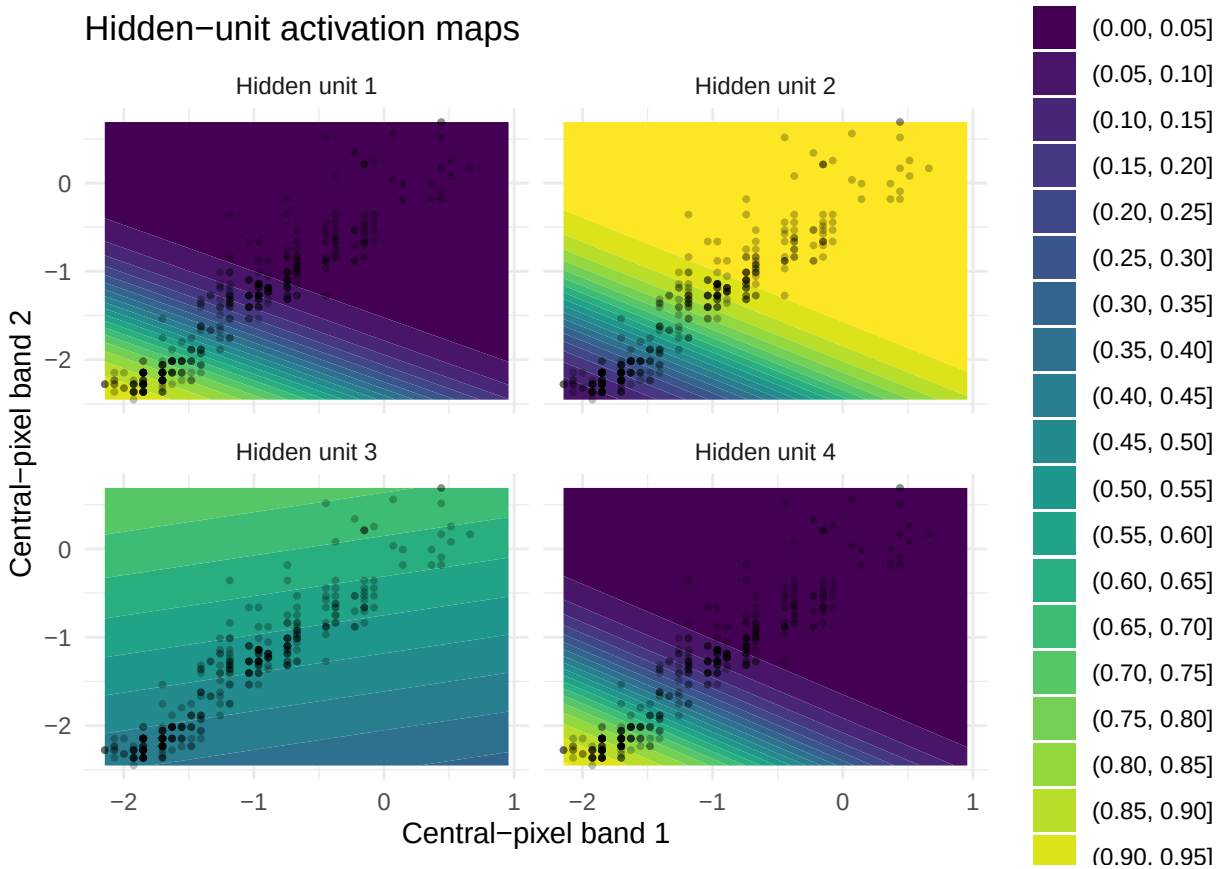
units_to_plot <- seq_len(
  min(
    4,
    ncol(hidden_activation_values)
  )
)

activation_plot_list <- lapply(
  units_to_plot,
  function(unit) {
    d <- prediction_grid$grid
    d$activation <-
      hidden_activation_values[
        ,
        unit
      ]
    d$unit <- paste(
      "Hidden unit",
      unit
    )
    d
  }
)

activation_plot_data <- do.call(
  rbind,
  activation_plot_list
)

ggplot(
  activation_plot_data,
  aes(
    x = band_1,
    y = band_2,
    z = activation
```

```
  )
) +
  geom_contour_filled(
    bins = 20
  ) +
  geom_point(
    data = test_plot_data,
    aes(
      x = band_1,
      y = band_2
    ),
    inherit.aes = FALSE,
    size = 0.7,
    alpha = 0.25
  ) +
  facet_wrap(
    ~ unit,
    ncol = 2
  ) +
  labs(
    title = "Hidden-unit activation maps",
    x = "Central-pixel band 1",
    y = "Central-pixel band 2",
    fill = "Activation"
  ) +
  theme_minimal()
```



11. Experiment together

Each row changes **one setting at a time** relative to the baseline neural network.

The important rule is: use the **validation set** to compare settings. Do not keep changing the model because one version happened to have a better test score.

```
evaluate_experiment <- function(
  label,
  hidden_units = 8,
  decay = 0.001,
  max_epochs = 200
) {
  set.seed(RANDOM_STATE)
  torch_manual_seed(
    RANDOM_STATE
  )

  parts <- build_network(
    hidden_units =
      hidden_units,
    decay = decay
  )

  fitted <- train_torch_model(
    parts$model,
    X_fit,
    y_fit,
    X_validation,
    y_validation,
    epochs = max_epochs,
    decay = parts$decay
  )

  validation_probability <- get_probability(
    fitted$model,
    X_validation
  )

  validation_pred <- as.integer(
    validation_probability >= 0.5
  )

  test_probability <- get_probability(
    fitted$model,
    X_test_scaled
  )

  test_pred <- as.integer(
    test_probability >= 0.5
  )

  data.frame(
    experiment = label,
    hidden_units = hidden_units,
```

```
    decay = decay,
    max_epochs = max_epochs,
    epochs_used =
      fitted$epochs_used,
    validation_accuracy =
      accuracy_score(
        y_validation,
        validation_pred
      ),
    validation_log_loss =
      binary_log_loss(
        y_validation,
        validation_probability
      ),
    test_accuracy =
      accuracy_score(
        y_test,
        test_pred
      ),
    test_log_loss =
      binary_log_loss(
        y_test,
        test_probability
      )
  )
}

experiment_settings <- list(
  list(
    label =
      "baseline: 8 hidden units",
    hidden_units = 8,
    decay = 0.001,
    max_epochs = 200
  ),
  list(
    label =
      "underfit: 1 hidden unit",
    hidden_units = 1,
    decay = 0.001,
    max_epochs = 200
  ),
  list(
    label =
      "larger: 32 hidden units",
    hidden_units = 32,
    decay = 0.001,
    max_epochs = 200
  ),
  list(
    label =
      "regularized: decay = 0.1",
    hidden_units = 8,
```

```
    decay = 0.1,
    max_epochs = 200
  ),
  list(
    label =
      "short training: 50 epochs",
    hidden_units = 8,
    decay = 0.001,
    max_epochs = 50
  ),
  list(
    label =
      "long training: 400 epochs",
    hidden_units = 8,
    decay = 0.001,
    max_epochs = 400
  )
)

experiment_results <- do.call(
  rbind,
  lapply(
    experiment_settings,
    function(s) {
      evaluate_experiment(
        label = s$label,
        hidden_units =
          s$hidden_units,
        decay = s$decay,
        max_epochs =
          s$max_epochs
      )
    }
  )
)

experiment_results
```

```
##                   experiment hidden_units
## 1  baseline: 8 hidden units            8
## 2   underfit: 1 hidden unit            1
## 3   larger: 32 hidden units           32
## 4  regularized: decay = 0.1            8
## 5 short training: 50 epochs            8
## 6 long training: 400 epochs            8
##   decay max_epochs epochs_used
## 1 0.001        200         200
## 2 0.001        200         200
## 3 0.001        200         200
## 4 0.100        200         200
## 5 0.001         50          50
## 6 0.001        400         400
##   validation_accuracy validation_log_loss
## 1           0.8716981           0.3749266
```

```
## 2           0.8641509           0.4397342
## 3           0.8716981           0.3537552
## 4           0.8490566           0.6503546
## 5           0.8528302           0.5733170
## 6           0.8716981           0.3519731
##   test_accuracy test_log_loss
## 1     0.9178470     0.2873894
## 2     0.9150142     0.3933331
## 3     0.9178470     0.2781792
## 4     0.9008499     0.6459840
## 5     0.9036827     0.5595058
## 6     0.9178470     0.2661727
```

Experiment decision boundaries

The first four experiments isolate changes in hidden-layer size and regularization. Compare the visual boundaries with the validation metrics.

```
fit_experiment_model <- function(
  hidden_units = 8,
  decay = 0.001,
  max_epochs = 200
) {
  set.seed(RANDOM_STATE)
  torch_manual_seed(
    RANDOM_STATE
  )

  parts <- build_network(
    hidden_units =
      hidden_units,
    decay = decay
  )

  fitted <- train_torch_model(
    parts$model,
    X_fit,
    y_fit,
    X_validation,
    y_validation,
    epochs = max_epochs,
    decay = parts$decay
  )

  fitted$model
}

boundary_settings <- experiment_settings[
  1:4
]

boundary_plot_list <- lapply(
  boundary_settings,
  function(settings) {
```

```
    model <- fit_experiment_model(
      hidden_units =
        settings$hidden_units,
      decay =
        settings$decay,
      max_epochs =
        settings$max_epochs
    )

    d <- prediction_grid$grid

    d$probability <- get_probability(
      model,
      prediction_grid$grid_scaled
    )

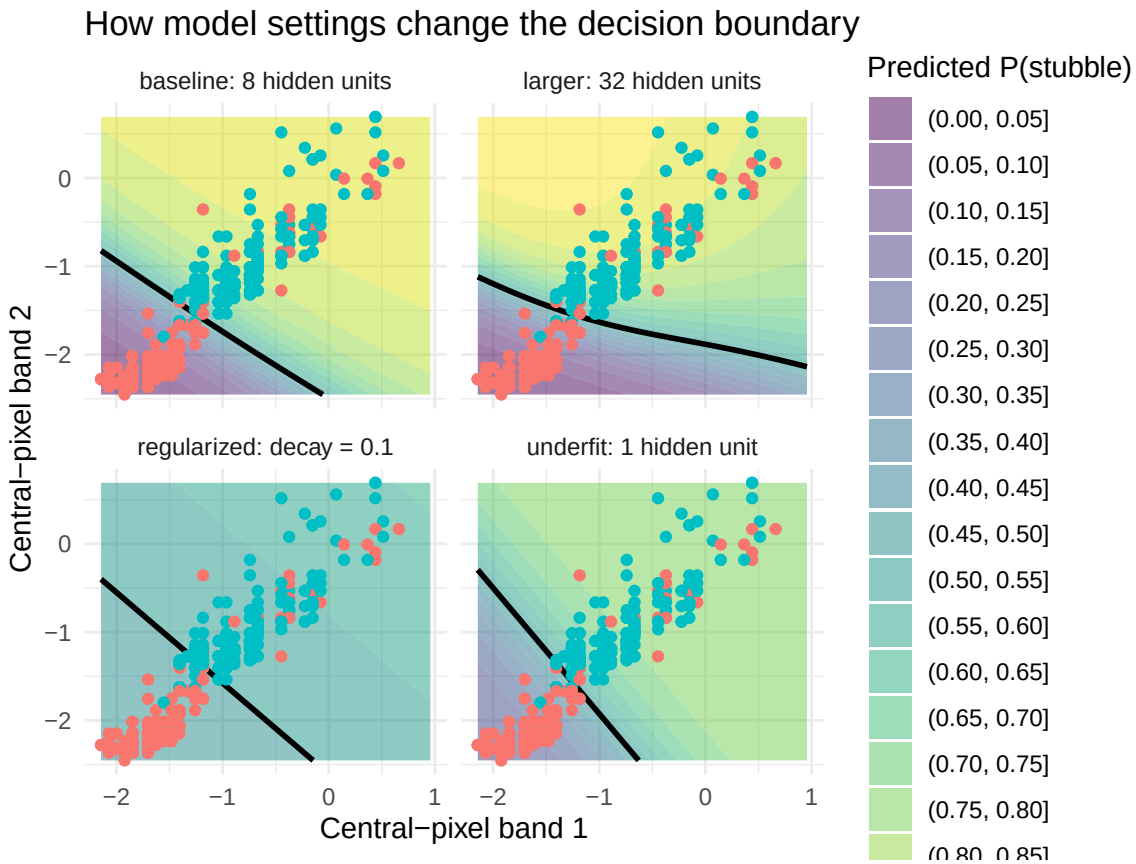
    d$experiment <-
      settings$label

    d
  }
)

boundary_plot_data <- do.call(
  rbind,
  boundary_plot_list
)

ggplot(
  boundary_plot_data,
  aes(
    x = band_1,
    y = band_2,
    z = probability
  )
) +
  geom_contour_filled(
    bins = 20,
    alpha = 0.5
  ) +
  geom_contour(
    breaks = 0.5,
    color = "black",
    linewidth = 1
  ) +
  geom_point(
    data = test_plot_data,
    aes(
      x = band_1,
      y = band_2,
      color = land_cover
    ),
    inherit.aes = FALSE,
```

```
  size = 1.5
) +
facet_wrap(
  ~ experiment,
  ncol = 2
) +
labs(
  title = "How model settings change the decision boundary",
  x = "Central-pixel band 1",
  y = "Central-pixel band 2",
  fill = "Predicted P(stubble)",
  color = "Observed test class"
) +
theme_minimal()
```



Exit questions

1. Why does a network need an activation function between dense layers? Activation functions allow for non-linear relationships between hidden units to relate the layers on the same plane of weights and biases.
2. What is an epoch, conceptually? One full run through of the model with the training data, more epochs continue to train the model but there should be diminishing payoffs at a certain point.
3. What does early stopping try to prevent? Overfitting by stopping the model from continuing to train as the validation error levels out.
4. Why can dropout reduce overfitting? It can train the model to not become overly reliant on one node within the hidden layer. If there is overreliance on one or a few nodes in the hidden layer, and they are

dropped the model will have to reevaluate the weights and biases of the other nodes.

5. What is a hidden unit doing here that logistic regression cannot do? Hidden units have more flexibility in the math that can shape the ultimate classification of the model with activation funstions and by adjusting weights and biases.